

Python

Summary Session 17



Преподаватель

Портрет

Имя Фамилия

Текущая должность

Количество лет опыта

Какой у Вас опыт - ключевые кейсы

Самые яркие проекты

Дополнительная информация по вашему усмотрению

Корпоративный e-mail

Социальные сети (по желанию)

Важно

-  Камера должна быть включена на протяжении всего занятия
-  В течение занятия вопросы задавать в чате или когда преподаватель спрашивает, есть ли у Вас вопросы
-  Вести себя уважительно и этично по отношению к остальным участникам занятия
-  Организационные вопросы по обучению решаются с кураторами, а не на тематических занятиях
-  Во время занятия будут интерактивные задания, будьте готовы включить камеру или демонстрацию экрана по просьбе преподавателя



ПОВТОРЕНИЕ ИЗУЧЕННОГО



Экспресс-опрос

?

Перечислите принципы ООП

Принципы ООП

Наследование

Один класс может **унаследовать** свойства и поведение другого. Это позволяет **избегать дублирования** кода и **переиспользовать логику**

Инкапсуляция

Объект **скрывает внутреннее устройство** и предоставляет **только нужный интерфейс**.
Позволяет защищать данные и контролировать доступ к ним

Полиморфизм

Одинаковый интерфейс может **вести себя по-разному** в зависимости от типа объекта. Позволяет использовать **одинаковый код для разных классов**

Абстракция

Выделение **общих характеристик и поведения**, чтобы задать интерфейс для будущих реализаций в наследниках

Знание принципов



Помогает понять, как правильно проектировать классы и объекты



Упрощают понимание механизмов наследования, переопределения и взаимодействия объектов



Позволяют писать гибкий, расширяемый и поддерживаемый код



Наследование

Это механизм, который позволяет одному классу перенять свойства и поведение другого. Это один из ключевых принципов ООП, который помогает избегать дублирования кода и повторно использовать общую логику.

Наследование

Родительский класс, базовый класс или суперкласс

Класс, от которого наследуются

Дочерний класс, производный класс или подкласс

Класс, который наследует

Экспресс-опрос

?

В чем состоит польза наследования?

Польза наследования



Избежание дублирования кода, путем выделения общего поведения в один базовый класс



Удобство расширять существующую логику без переписывания старых классов



Работа с разными объектами одинаковым образом, если они наследуются от общего предка

Наследование: синтаксис

```
class Parent:  
    # родительский класс  
    ...  
  
class Child(Parent):  
    # дочерний класс, наследует всё от Parent  
    ...
```

Наследование



Пример

```
class Employee:
    def __init__(self, name):
        self.name = name

    def work(self):
        print(f"{self.name} is working...")

class Programmer(Employee):
    pass

class Manager(Employee):
    pass

programmer = Programmer("Alice")
manager = Manager("Bob")

programmer.work()
manager.work()
```

Пояснение

- Employee — базовый класс, который хранит имя сотрудника и умеет работать.
- Programmer и Manager **наследуют всё от Employee**: и поле name, и метод work().
- Каждый из них может использовать эти возможности **без повторного определения**.

Особенности



Дочерний класс наследует все поля и методы родительского класса



Можно добавлять свои методы и поля, не затрагивая родителя



Можно переопределять поведение, если нужно изменить работу унаследованного метода



Полиморфизм

(от греч. «много форм»)

Это возможность использовать один и тот же интерфейс для разных типов объектов.

Экспресс-опрос

?

В чем состоит польза полиморфизма?

Польза полиморфизма



Позволяет писать универсальный код, не заботясь о типе объекта



Упрощает расширение программы — можно добавлять новые типы, не меняя старую логику



Делает код гибким и читаемым

Механизмы полиморфизма



Полиморфизм в Python реализуется через переопределение методов (overriding), когда дочерний класс заново определяет метод, унаследованный от родителя. При вызове такого метода используется реализация из дочернего класса.

Пример: переопределение метода

```
class Employee:

    def __init__(self, name):
        self.name = name

    def work(self):
        print(f"{self.name} is working...")

class Programmer(Employee):
    def work(self):
        print(f"{self.name} writing code...")

class Manager(Employee):
    def work(self):
        print(f"{self.name} managing team...")

staff = [Programmer("Alice"), Manager("Bob"), Programmer("Bill")]

for person in staff:
    person.work() # Поведение зависит от конкретного типа
```

- У всех сотрудников есть метод `work()`, но он реализован по-разному.
- В списке `staff` мы храним объекты разных классов: `Programmer`, `Manager`.
- При вызове `person.work()` Python **автоматически вызывает нужную версию метода** в зависимости от того, к какому классу относится объект.
- Это и есть **полиморфизм: один вызов → разное поведение в зависимости от типа объекта**.

Экспресс-опрос

?

Какая функция вызывает ближайший метод родителя и используется, чтобы не дублировать одинаковый код?

Важно



При наследовании часто возникает ситуация, когда родительский класс уже задаёт общие поля в `__init__()`, а в дочернем классе необходимо добавить новые поля. Чтобы **не дублировать одинаковый код** используется функция `super()` — она вызывает ближайший метод родителя.

Пример

```
class Employee:
    def __init__(self, name):
        self.name = name

class Programmer(Employee):
    def __init__(self, name, language):
        super().__init__(name) # вызываем родительский __init__
        self.language = language

class Manager(Employee):
    def __init__(self, name, department):
        super().__init__(name) # вызываем родительский __init__
        self.department = department

p = Programmer("Alice", "Python")
print(p.name)
print(p.language)
```

Функция `super` в обычных методах



Функция `super` полезна не только в `__init__()`, но и **в любых других методах**, где нужно **расширить поведение родителя**, а не полностью его заменить.

Пример: расширение обычного метода

```
class Employee:
    def work(self):
        print("Employee is doing general tasks.")

class Programmer(Employee):
    def work(self):
        super().work() # вызываем метод родителя
        print("Programmer is writing code.")

class Manager(Employee):
    def work(self):
        super().work()
        print("Manager is holding a meeting.")

staff = [Programmer(), Manager()]
for person in staff:
    person.work()
    print()
```


Эффективность super



Автоматически находит метод в ближайшем родителе



Позволяет избежать дублирования кода



Гарантирует корректную работу при изменении иерархии классов

Экспресс-опрос

?

В каких случаях используется функция super?

Использование super



Когда **родитель** уже делает нужную инициализацию, и мы хотим её сохранить

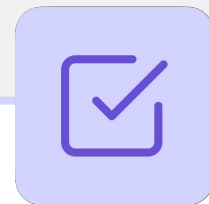


Когда **расширяем** (а не полностью заменяем) поведение метода



Особенно важно при **множественном наследовании** — `super()` автоматически учитывает порядок вызова

Важно



Python предоставляет удобные встроенные функции для **проверки типов и иерархии классов**, что особенно полезно при работе с **наследованием** и **полиморфизмом**.

Экспресс-опрос

A small purple square icon containing a white question mark.

Какая встроенная функция Python позволяет проверить, является ли объект экземпляром определённого класса или его подкласса?



isinstance

Это встроенная функция Python, которая позволяет проверить, является ли объект экземпляром определённого класса или его подкласса, чтобы выбрать правильное поведение

Особенности isinstance



Если объект принадлежит **указанному классу или его наследнику**, функция возвращает True, в противном случае — False.

Функция `isinstance` также работает со **встроенными типами** (`str`, `list`, `int` и т.д.)

Функция isinstance



Синтаксис

```
isinstance(obj, some_class)
isinstance(obj, tuple_of_classes)
```

Пояснение

- `obj` — объект, который необходимо проверить
- `some_class` — класс, принадлежность к которому необходимо проверить
- `tuple_of_classes` — кортеж из нескольких классов, принадлежность к любому из которых необходимо проверить

Пример

```
class Employee:
    pass

class Programmer(Employee):
    pass

class Manager(Employee):
    pass

e = Employee()
p = Programmer()
m = Manager()

print(isinstance(p, Programmer)) # экземпляр класса
print(isinstance(p, Employee))  # экземпляр наследника
print(isinstance(p, Manager))   # Manager не находится выше в иерархии
print(isinstance(p, object))    # True – все классы наследуют от object
```

Пример использования в коде

```
class Employee:
    def work(self):
        print("Выполняет общие задачи")

class Programmer(Employee):
    def write_code(self):
        print("Пишет код")

class BackendDeveloper(Programmer):
    def write_code(self):
        print("Пишет серверный код")

class FrontendDeveloper(Programmer):
    def write_code(self):
        print("Пишет интерфейс")
```

1

```
class Manager(Employee):
    def work(self):
        print("Проводит собрание")

staff = [
    Programmer(),
    BackendDeveloper(),
    FrontendDeveloper(),
    Manager(),
    Employee()
]

for person in staff:
    if isinstance(person, Programmer):
        person.write_code()
    else:
        person.work()
```

2

Проверка на несколько классов



Пример

```
print(isinstance("hello", (str, int))) #  
Строка принадлежит к одному из классов
```

Пояснение

Можно проверить, принадлежит ли объект **к одному из нескольких классов**, передав кортеж

Экспресс-опрос

A small purple square icon containing a white question mark.

Какая встроенная функция Python позволяет проверить, является ли один класс подклассом другого?



issubclass

Это встроенная функция Python, которая позволяет проверить, является ли один класс подклассом другого, то есть наследуется ли он от указанного класса

Особенности isinstance



Функция возвращает `True`, если класс **наследуется от другого класса напрямую или через цепочку**, и `False` — если нет.

Функция `issubclass`



Синтаксис

```
issubclass(class_a, class_b)
issubclass(class_a, tuple_of_classes)
```

Пояснение

- `class_a` — класс, который мы проверяем
- `class_b` — предполагаемый родительский класс
- `tuple_of_classes` — кортеж из нескольких классов

Пример

```
class Employee:
    pass

class Programmer(Employee):
    pass

class Manager(Employee):
    pass

class BackendDeveloper(Programmer):
    pass

print(issubclass(Programmer, Employee)) # Прямой потомок
print(issubclass(BackendDeveloper, Programmer)) # Прямой потомок
print(issubclass(BackendDeveloper, Employee)) # Потомок через цепочку
print(issubclass(Manager, Programmer)) # Разные ветки иерархии
print(issubclass(Employee, object)) # Все классы наследуют от object
```


Экспресс-опрос

?

Как называется корневой класс всей иерархии классов в Python?



Класс object

Это корневой класс всей иерархии классов в Python. Он предоставляет набор базовых методов, таких как `__str__()`, `__init__()` и другие.

Класс object



Пример

```
class Book:
    pass
```

Пояснение

Python воспринимает это так:

```
class Book(object):
    pass
```

Существующие в классе методы



Пример

```
print(dir(Book))
```

Пояснение

Список уже существующих в классе методов можно увидеть с помощью функции `dir()`

Магический метод `__str__`



Пример

```
b = Book()

print(b.__str__()) # Вызов метода
__str__ напрямую
print(b)           # Вызов метода
__str__
```

Пояснение

Когда объект выводится через `print()`, Python вызывает магический метод `__str__()`, чтобы получить **строковое представление объекта**, которое будет показано пользователю

Экспресс-опрос

?

Назовите особенности метода `__str__`

Особенности метода `__str__`



Если метод `__str__()` не определён, Python выводит **техническую информацию**



Определение метода `__str__()` позволяет **вернуть читаемую строку** о содержимом объекта



Не должен ничего выводить сам — **только возвращать строку**



Вызывается автоматически при `print(obj)` или `str(obj)`

Пример: техническая информация

```
class Book:  
    def __init__(self, title, author):  
        self.title = title  
        self.author = author
```

```
book = Book("1984", "George Orwell")  
print(book)
```


Пример: содержимое объекта

```
class Book:
    def __init__(self, title, author):
        self.title = title
        self.author = author

    def __str__(self):
        return f"{self.title} by {self.author}"

book = Book("1984", "George Orwell")
print(book)           # вызывает __str__
print(str(book))      # вызывает __str__
print(book.__str__()) # вызывает __str__
```

Экспресс-опрос

?

Назовите преимущества наследования от object

Преимущества наследования от object



Все классы получают **единое поведение по умолчанию**



Все объекты можно безопасно использовать с базовыми функциями и методами



Это делает классы совместимыми со встроенными механизмами Python (например, `print()` вызывает `__str__()` из `object`, если не переопределено)



Декораторы

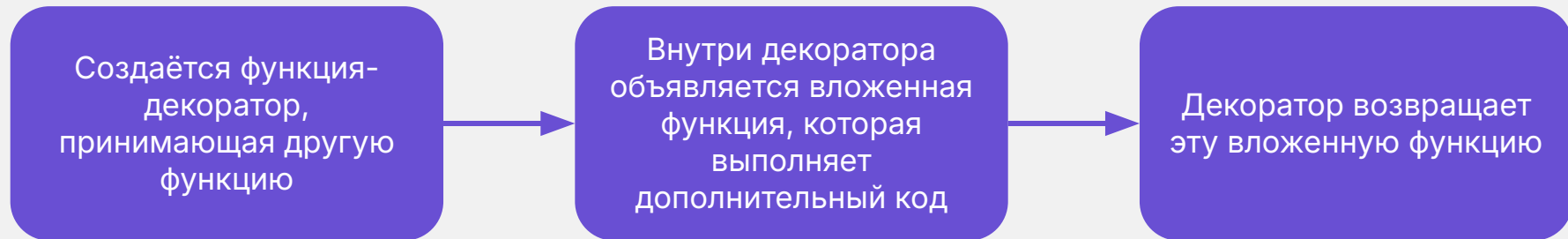
Это способ изменить поведение функции, не изменяя её код. Декоратор принимает функцию, добавляет к ней новую логику и возвращает изменённую версию

Экспресс-опрос

?

Как работает декоратор?

Работа декоратора



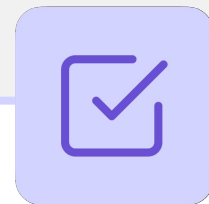
Пример

```
def simple_decorator(func):    # Функция-декоратор, принимает другую функцию
    def wrapper():            # Вложенная функция-обертка, добавляющая дополнительное поведение
        print("Перед вызовом функции")
        func()                # Вызываем переданную функцию
        print("После вызова функции")
    return wrapper            # Возвращаем изменённую функцию

def say_hello():
    print("Привет!")

decorated = simple_decorator(say_hello)    # Вызываем декоратор, теперь decorated = wrapper
print(decorated)
decorated()    # Теперь вызов say_hello() происходит через wrapper
```

wrapper



После применения декоратора результат (wrapper) можно сохранить в переменной с тем же именем, что и декорируемая функция. Тогда вместо вызова оригинальной функции будет вызываться функция wrapper, добавляющая дополнительный функционал.

Пример

```
def simple_decorator(func):    # Функция-декоратор, принимает другую функцию
    def wrapper():            # Вложенная функция-обертка, добавляющая дополнительное поведение
        print("Перед вызовом функции")
        func()                # Вызываем переданную функцию
        print("После вызова функции")
    return wrapper            # Возвращаем изменённую функцию

def say_hello():
    print("Привет!")

say_hello = simple_decorator(say_hello)    # Вызываем декоратор, теперь decorated = wrapper
print(say_hello)
say_hello()    # Теперь вызов say_hello() происходит через wrapper
```

Экспресс-опрос

?

Как можно вызвать декоратор?

Синтаксис @decorator



Вместо явного вызова `decorated = simple_decorator(say_hello)` можно использовать `@` с именем декоратора перед определением функции

Пример

```
def simple_decorator(func):    # Функция-декоратор, принимает другую функцию
    def wrapper():            # Вложенная функция, добавляющая дополнительное поведение
        print("Перед вызовом функции")
        func()                # Вызываем переданную функцию
        print("После вызова функции")
    return wrapper            # Возвращаем изменённую функцию

@simple_decorator              # Эквивалентно say_hello = simple_decorator(say_hello)
def say_hello():
    print("Привет!")

say_hello()
```

Применение декоратора



Декораторы полезны, когда нужно добавлять дополнительное поведение к функциям. Один из распространённых примеров — декоратор для измерения времени выполнения.

Пример

```
import time

def timing_decorator(func):
    def wrapper():
        start_time = time.time() # Засекаем время перед выполнением функции
        func() # Вызываем декорируемую функцию
        end_time = time.time() # Засекаем время после выполнения
        print(f"Функция {func.__name__} выполнялась {end_time - start_time:.5f} секунд")
    return wrapper

@timing_decorator # Применение декоратора
def slow_function():
    time.sleep(2) # Имитация долгой операции
    print("Функция выполнена")

slow_function()
```

Экспресс-опрос

?

Где применяются декораторы?

Основные области применения декораторов



Логирование вызовов функций



Измерение времени выполнения



Ограничение частоты вызовов
(Throttling)



Проверка и валидация входных
данных



Автоматическое повторение
выполнения (Retry)



Ограничение доступа



Кеширование результатов



Автоматическое изменение данных

Экспресс-опрос



Что такое поля класса?



Поле класса (атрибут класса)

Это переменная, которая принадлежит самому классу, а не отдельному объекту. Все экземпляры класса могут получить к ней доступ, и при этом значение хранится в одном месте — внутри класса.

Поле класса



Синтаксис

```
class ClassName:  
    class_attribute = value # поле  
класса
```

Пояснение

- `class_attribute` — поле класса, общее для всех объектов
- `value` — значение этого поля

Пример

```
class Book:
    library_name = "Central Library" # поле класса

    def __init__(self, title, author):
        self.title = title # поле объекта
        self.author = author # поле объекта
```

Использование полей класса



Для хранения настроек, категорий, счётчиков, общих значений



Когда значение одинаково для всех объектов

Изменение полей класса



Поля класса можно читать и изменять как внутри класса, так и снаружи — через класс или объект. Но при этом поведение может отличаться — особенно при изменении значений.

Экспресс-опрос



Как можно читать поля класса?

1. Чтение поля класса



Напрямую через класс: `ClassName.attribute`



Через объект: `object_name.attribute`



Внутри методов класса — через `self.attribute` или `ClassName.attribute`

Пример чтения поля класса

```
class Book:
    library_name = "Central Library" # поле класса

    def __init__(self, title, author):
        self.title = title
        self.author = author

    def show_library(self):
        print(self.library_name) # доступ через self
        print(Book.library_name) # доступ через имя класса

print(Book.library_name) # доступ через класс
book = Book("1984", "George Orwell")
print(book.library_name) # доступ через объект
book.show_library()      # доступ изнутри
```

2. Изменение значения через класс

```
class Book:
    library_name = "Central Library" # поле класса

    def __init__(self, title, author):
        self.title = title
        self.author = author

book = Book("1984", "George Orwell")
book2 = Book("Brave New World", "Aldous Huxley")
Book.library_name = "City Library"
print(Book.library_name)
print(book.library_name)      # у объекта тоже изменилось
print(book2.library_name)     # у объекта тоже изменилось
```

3. Создание поля через объект



Если попытаться изменить значение поля через объект, на самом деле будет создано **новое поле в конкретном объекте**, не затронув поле класса.

Пример создания поля через объект

```
class Book:
    library_name = "Central Library" # поле класса

    def __init__(self, title, author):
        self.title = title
        self.author = author

book = Book("1984", "George Orwell")
book2 = Book("Brave New World", "Aldous Huxley")

book.library_name = "Private Shelf" # создаёт новое поле в book
print(book.library_name) # Private Shelf – новое значение в объекте
print(Book.library_name) # поле класса не изменилось
print(book2.library_name) # также прочитает поле класса
```

Экспресс-опрос

?

Что такое классовой метод?



Классовый метод

Это метод, который работает не с конкретным объектом, а с самим классом. Он получает доступ не к `self`, а к `cls` — ссылке на класс, из которого был вызван.

Важно



Чтобы объявить классовый метод, используется декоратор `@classmethod`.

Применение классовых методов



Работа с полями класса (например, счётчиками или общими настройками)



Добавление поведения, относящегося к самому классу, а не к отдельному экземпляру

Классовые методы



Синтаксис

```
class ClassName:  
    @classmethod  
    def method_name(cls, ...):  
        ...
```

Пояснение

- `@classmethod` — декоратор, помечающий метод как классový
- `cls` — ссылка на сам класс (аналог `self`, но для класса)

Пример: счётчик созданных объектов

```
class Book:
    total_books = 0 # поле класса

    def __init__(self, title, author):
        self.title = title
        self.author = author
        Book.total_books += 1

    @classmethod
    def show_total(cls):
        print(f"Total books: {cls.total_books}")
        # print(cls.title) # Нельзя обратиться к полю объекта

Book.show_total() # Вызов через класс
book1 = Book("1984", "George Orwell")
book2 = Book("Brave New World", "Aldous Huxley")

Book.show_total() # Вызов через класс
book1.show_total() # Вызов через объект
```

Пример: поведение, связанное с классом

```
class Book:
    default_format = "PDF"
    default_language = "en"

    @classmethod
    def show_defaults(cls):
        print(f"Format: {cls.default_format}, Language: {cls.default_language}")

    @classmethod
    def reset_defaults(cls):
        cls.default_format = "PDF"
        cls.default_language = "en"

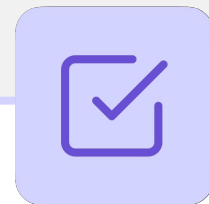
Book.default_format = "EPUB" # Изменение формата
Book.show_defaults()         # Просмотр текущих настроек
Book.reset_defaults()        # Сброс до базовых настроек
Book.show_defaults()         # Просмотр текущих настроек
```



Статический метод

Это метод, который не зависит ни от объекта, ни от самого класса. Он работает как обычная функция, но помещён внутрь класса для логической связанности с ним.

Важно



Чтобы объявить статический метод, используется декоратор `@staticmethod`

Экспресс-опрос

?

Для чего используются статические методы?

Использование статических методов



Когда функция логически относится к классу, но не использует ни `self`, ни `cls`



Чтобы сгруппировать связанную логику внутри класса, а не выносить её в отдельные функции вне класса

Статические методы



Синтаксис

```
class ClassName:  
    @staticmethod  
    def method_name(...):  
        ...
```

Пояснение

- `@staticmethod` — декоратор, превращающий метод в статический
- Такой метод **не получает ни self, ни cls**

Пример: вспомогательная функция внутри класса

```
class Book:
    def __init__(self, title):
        self.title = title

    @staticmethod
    def is_title_valid(title):
        return isinstance(title, str) and len(title) > 0

print(Book.is_title_valid("1984"))      # True
print(Book.is_title_valid(""))         # False
print(Book.is_title_valid(123))        # False
```

Такой метод можно вызывать и через класс (`Book.is_title_valid()`), и через объект — результат будет одинаков

Пример: логически связанные, но независимые операции

```
class MathHelper:
    @staticmethod
    def square(x):
        return x * x

    @staticmethod
    def cube(x):
        return x * x * x

print(MathHelper.square(5))
print(MathHelper.cube(3))
```

Сравнение типов методов

Тип метода	Обычный метод	Классовый метод	Статический метод
Декоратор	–	@classmethod	@staticmethod
Первый аргумент	self	cls	–
Доступ	К полям и методам объекта	К полям и методам класса	–
Когда использовать	Когда метод работает с конкретным экземпляром	Когда метод работает с классом или создаёт объекты по шаблону	Когда функция логически относится к классу, но ничего не использует



ВОПРОСЫ





ЗАДАНИЕ

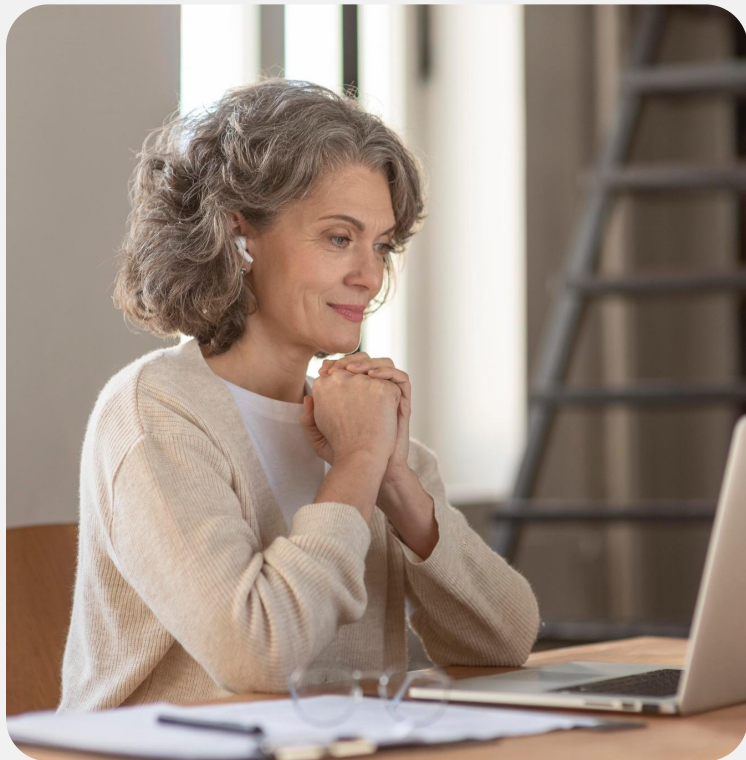




Выполните задание

1.1 Сопоставьте понятие с его описанием:

1. Наследование
 2. Полиморфизм
 3. Инкапсуляция
 4. Абстракция
-
- a. Скрытие внутренней реализации и предоставление только нужного интерфейса
 - b. Поведение зависит от конкретного подкласса
 - c. Выделение общих характеристик для дальнейшей реализации
 - d. Повторное использование кода из базового класса



Выполните задание

1.1 Сопоставьте понятие с его описанием:

1. Наследование
 2. Полиморфизм
 3. Инкапсуляция
 4. Абстракция
-
- a. Скрытие внутренней реализации и предоставление только нужного интерфейса
 - b. Поведение зависит от конкретного подкласса
 - c. Выделение общих характеристик для дальнейшей реализации
 - d. Повторное использование кода из базового класса

Ответ: 1-d, 2-b, 3-a, 4-c



Выберите верный вариант ответа

1.2 Что делает функция `super()` в классе-наследнике?

- a. Обращается к родителю по имени напрямую
- b. Создаёт экземпляр родительского класса
- c. Автоматически вызывает метод родителя, учитывая порядок наследования
- d. Удаляет метод родителя



Выберите верный вариант ответа

1.2 Что делает функция `super()` в классе-наследнике?

- a. Обращается к родителю по имени напрямую
- b. Создаёт экземпляр родительского класса
- c. Автоматически вызывает метод родителя, учитывая порядок наследования
- d. Удаляет метод родителя



Ответьте на вопрос

1.3 Что вернёт функция isinstance() и почему?

```
class Animal:
    pass
```

```
class Dog(Animal):
    pass
```

```
d = Dog()
print(isinstance(d, Animal))
```



Ответьте на вопрос

1.3 Что вернёт функция `isinstance()` и почему?

```
class Animal:
    pass

class Dog(Animal):
    pass

d = Dog()
print(isinstance(d, Animal))
```

Ответ: True, потому что Dog наследует от Animal



Ответьте на вопрос

1.4 Что напечатает код?

```
class A:
    pass
```

```
class B(A):
    pass
```

```
b = B()
print(isinstance(b, B))
print(isinstance(b, A))
print(isinstance(b, object))
```



Ответьте на вопрос

1.4 Что напечатает код?

```
class A:
    pass
```

```
class B(A):
    pass
```

```
b = B()
print(isinstance(b, B))
print(isinstance(b, A))
print(isinstance(b, object))
```

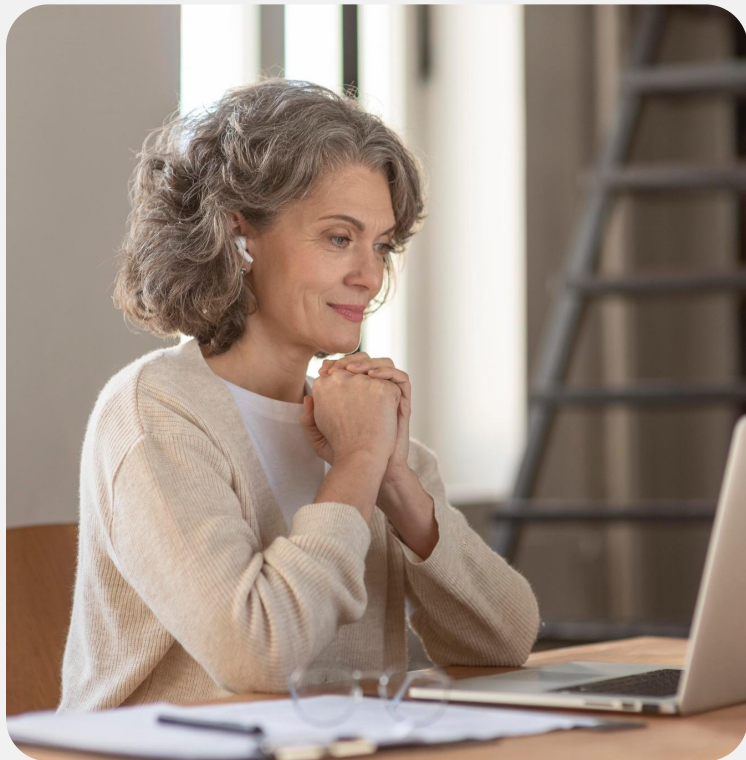
Ответ:

True
True
True



ВОПРОСЫ





Выполните задание

2.1 Найдите ошибку в декораторе:

```
def my_decorator(func):
    def wrapper:
        print("До")
        func()
        print("После")
    return wrapper
```

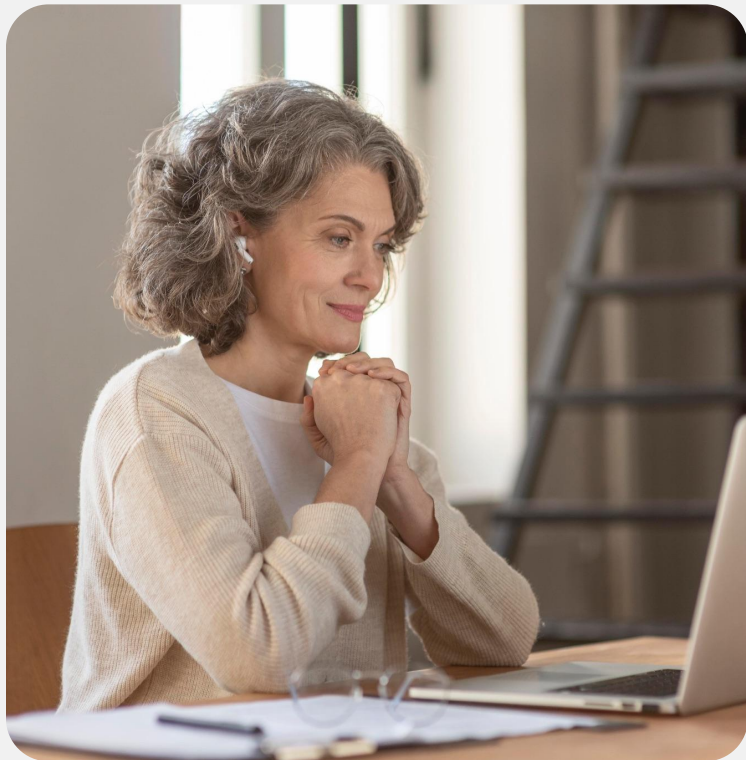



Выполните задание

2.1 Найдите ошибку в декораторе:

```
def my_decorator(func):
    def wrapper:
        print("До")
        func()
        print("После")
    return wrapper
```

Ответ: Пропущены скобки после def wrapper



Выполните задание

2.2 Сопоставьте часть декоратора с его назначением:

1. `def decorator(func)`
 2. `def wrapper()`
 3. `func()`
 4. `return wrapper`
-
- a. Определение вложенной функции
 - b. Возврат новой функции вместо оригинальной
 - c. Вызов исходной функции внутри новой
 - d. Получение функции как аргумента



Выполните задание

2.2 Сопоставьте часть декоратора с его назначением:

1. `def decorator(func)`
 2. `def wrapper()`
 3. `func()`
 4. `return wrapper`
-
- a. Определение вложенной функции
 - b. Возврат новой функции вместо оригинальной
 - c. Вызов исходной функции внутри новой
 - d. Получение функции как аргумента

Ответ: 1-d, 2-a, 3-c, 4-b



Ответьте на вопрос



2.3 Что произойдёт при выполнении?

```
class Book:
    pass
```

```
book = Book()
book.author = "Orwell"
print(book.author)
```



Ответьте на вопрос

2.3 Что произойдёт при выполнении?

```
class Book:
    pass
```

```
book = Book()
book.author = "Orwell"
print(book.author)
```

Ответ: создаст поле `author` у конкретного объекта `book` и выведет `Orwell`



Ответьте на вопрос

2.3 Что произойдёт при выполнении?

```
class Book:
    pass
```

```
book = Book()
book.author = "Orwell"
print(book.author)
```

Ответ: создаст поле author у конкретного объекта book и выведет Orwell

Выберите верный вариант ответа

2.4 Что выведет код?

```
class Book:
    library_name = "Central Library"

    def __init__(self, title):
        self.title = title

book = Book("1984")
book.library_name = "Private Shelf"
print(book.library_name)
print(Book.library_name)
```

- a. Private Shelf
- Central Library
- b. Central Library
- Library
- c. Private Shelf
- Private Shelf

Выберите верный вариант ответа

2.4 Что выведет код?

```
class Book:
    library_name = "Central Library"

    def __init__(self, title):
        self.title = title

book = Book("1984")
book.library_name = "Private Shelf"
print(book.library_name)
print(Book.library_name)
```

- a. Private Shelf
Central Library
- b. Central Library
Central Library
- c. Private Shelf
Private Shelf



Выполните задание

2.5 Сопоставьте тип метода с его описанием:

1. Обычный метод
 2. Классовый метод
 3. Статический метод
-
- a. Получает `cls` и работает с классом
 - b. Получает `self` и работает с конкретным объектом
 - c. Не получает `self` или `cls`, работает как обычная функция



Выполните задание

2.5 Сопоставьте тип метода с его описанием:

1. Обычный метод
 2. Классовый метод
 3. Статический метод
-
- a. Получает `cls` и работает с классом
 - b. Получает `self` и работает с конкретным объектом
 - c. Не получает `self` или `cls`, работает как обычная функция

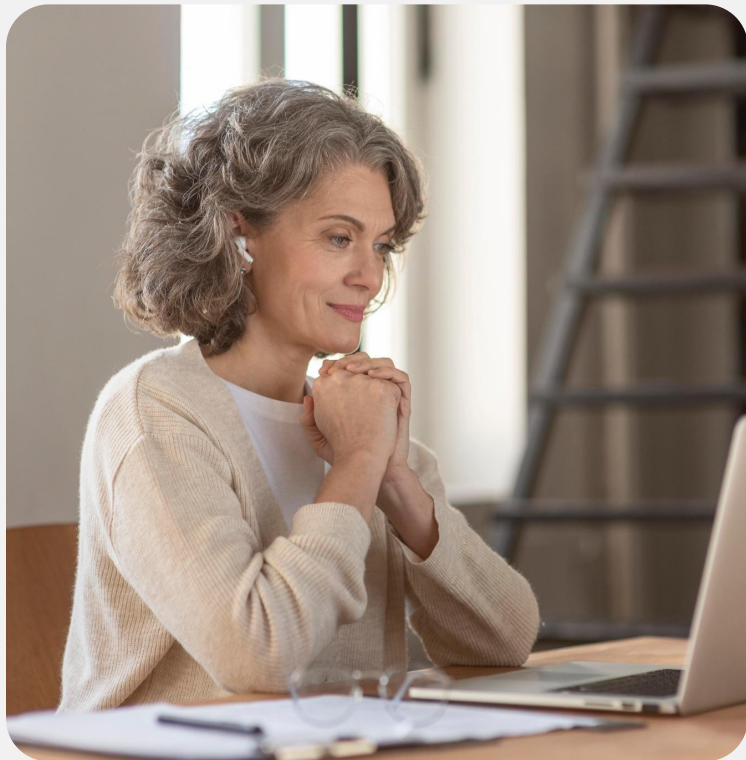
Ответ: 1-b, 2-a, 3-c



Выполните задание

2.6 Найдите ошибку в коде:

```
class Book:
    @classmethod
    def info():
        print("Книга")
```



Выполните задание

2.6 Найдите ошибку в коде:

```
class Book:
    @classmethod
    def info():
        print("Книга")
```

Ответ: отсутствует аргумент cls



ВОПРОСЫ





**Доработайте
практические
задания**



1. Класс Student



Создайте класс `Student`, представляющий студента.

- У каждого объекта должно быть имя и дата рождения.
- Дата передаётся как строка (YYYY-MM-DD) и сохраняется в виде объекта `datetime.date`.
- Добавьте проверку возраста — студенту должно быть не менее 16 лет, иначе выбрасывается `ValueError`.

2. Строковое представление



Добавьте строковое представление объекта.

Пример вывода:

Student: Alice, birth_date: 2005-05-10

3. Возраст на конкретную дату



Добавьте возможность получить возраст студента на переданную дату.

4. Карточка студента

Добавьте метод, который выводит информацию о студенте на текущий момент.

- Возраст вычисляется автоматически на основе даты рождения.

Пример вывода:

Student:

Name: Alice

Age: 17

5. Фильтрация студентов по возрасту

Реализуйте способ отобрать из списка студентов только тех, кто старше определённого возраста.

6. Проверка работы класса

- Создайте нескольких студентов.
- Выведите информацию о каждом.
- Попробуйте передать некорректную дату и отловите исключение.
- Получите список студентов старше указанного возраста.

Пример вывода:

Student:

Name: Alice

Age: 17

Student:

Name: Bob

Age: 21

Students older than 18:

Student: Bob, birth_date: 2003-07-22

7. Дочерний класс BachelorStudent

На основе класса Student создайте класс BachelorStudent, добавьте в него новые поля:

- `year_of_study`: текущий год обучения (например, 1-й курс, 2-й курс и т.д.).
- `major`: основная специализация или направление обучения.

8. Дочерний класс MasterStudent



На основе класса Student создайте класс MasterStudent, добавьте в него новые поля:

- `research_topic`: тема исследования или диссертации.
- `advisor`: научный руководитель.

9. Переопределение метода `student_card()`

Переопределите метод `student_card`, чтобы включить дополнительную информацию дочерних классов.

10. Проверка работы классов

- Создайте нескольких студентов разных классов.
- Выведите информацию о каждом с указанием типа студента.

Пример вывода:

Bachelor Student:

Name: Anna
 Age: 19
 Year of Study: 3
 Major: Computer Science

Student:

Name: Alice
 Age: 17

Master Student:

Name: Ben
 Age: 23
 Research Topic: Machine Learning
 Advisor: Dr. Smith



РАЗБОР ДОМАШНЕГО ЗАДАНИЯ



Домашнее задание

1. Класс Person

Создайте класс Person, представляющий человека.

- Каждый человек должен иметь имя.
- Добавьте метод `introduce()`, который выводит приветствие с именем.

Пример вывода:

Hello, my name `is` Alice.

Домашнее задание

2. Класс Student

На основе класса `Person` создайте класс `Student`.

- Студент должен иметь имя и номер курса.
- Метод `introduce()` должен сначала выводить базовое приветствие, а затем строку: `I'm on course <номер_курса>`.

Пример вывода:

```
Hello, my name is Alice.
```

```
I'm on course 2.
```

Домашнее задание

3. Класс Teacher и список людей

На основе класса Person создайте класс Teacher.

- У преподавателя есть имя и предмет.
- Метод `introduce()` должен выводить имя и предмет.
- Метод `introduce()` должен выводить строку: `Hello, I am professor <имя>. My subject is <предмет>.`
- Создайте список, в котором будут `Student` и `Teacher`, и вызовите у всех метод `introduce()`.

Пример вывода:

```
Hello, my name is Alice.
I'm on course 2.
Hello, I am professor Bob.
My subject is Mathematics
```



ВОПРОСЫ



Домашнее задание

1. Счётчик экземпляров

Создайте класс `User`, представляющий пользователя.

- При создании должны указываться логин (`username`) и пароль (`password`).
- У класса должно быть поле `total_users`, хранящее общее количество созданных пользователей.
- При каждом создании нового объекта `User`, счётчик должен увеличиваться.
- Добавьте метод `get_total()`, возвращающий количество пользователей.
- Проверьте, что счётчик работает.

Пример вывода:

```
Total users: 2
```

Домашнее задание

2. Проверка данных пользователя

Доработайте класс User.

- Добавьте валидации полей при создании.
- Имя должно быть непустой строкой.
- Пароль должен быть строкой длиной не менее 5 символов.
- Если данные некорректны — выбрасывайте ValueError.
- Добавьте строковое представление объекта.
- Проверьте работу класса с разными значениями.

Пример вызова:

```
user1 = User("alice", "secret")
user2 = User("bob", "qwe")
```

Пример вывода:

```
User: alice
...
ValueError: Invalid password: 'qwe'.
```



ВОПРОСЫ



Заключение

